

Training Deep Neural Networks

Hands-On Machine Learning (Aurelien Geron)

Giacomo Fiumara

Roadmap

- Deep neural networks are hard to train.
- Unstable gradients
- Initialization
- Activation functions
- Batch normalization, gradient clipping, transfer learning, unsupervised pretraining, faster optimizers, learning-rate schedules, and regularization.

The vanishing/exploding gradients problem

Why deep networks are harder

- A deep neural network has many layers, so information and gradients must pass through many transformations.
- This can make training unstable, slow, data-hungry, and more likely to overfit.

Main difficulties in deep learning

- Gradients may vanish or explode, so early layers learn very slowly or training diverges.
- Large models need lots of data and computation, but labeled data is often expensive.
- Millions of parameters can overfit unless optimization and regularization are handled carefully.

Vanishing gradients

- Backpropagation sends error information from the output layer back toward the input layer.
- If gradients keep shrinking as they travel backward, lower layers receive almost no useful update.
- The result is that the network trains badly because the first layers remain almost unchanged.

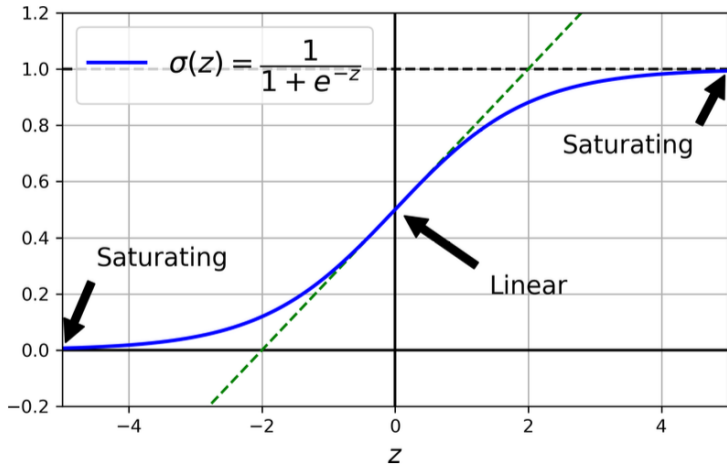
Exploding gradients

- The opposite problem happens when gradients grow larger and larger as they propagate backward.
- Then weight updates become too large, so optimization becomes unstable and may diverge.
- This is especially common in recurrent networks, but deep feedforward nets can suffer too.

Why sigmoid caused trouble

- The sigmoid activation saturates near 0 and 1, where its slope is extremely small.
- When a layer is saturated, backpropagation has almost no gradient to pass to previous layers.
- This is one reason why early deep networks with sigmoid units were hard to train.

Sigmoid



Signal flow intuition

- Good training requires activations and gradients to keep a reasonable scale from layer to layer.
- If signals grow too much, later units saturate or explode; if they shrink too much, learning fades away.
- Initialization and activation functions are therefore central design choices in deep learning.

Glorot and He initialization

- A key idea is to initialize weights so that the variance of signals stays controlled across layers.
- The variance of the outputs of each layer must be equal to the variance of its inputs.
- The gradients must have equal variance before and after flowing through a layer in the reverse direction.
- He initialization is designed for ReLU and related activations, which behave differently on the positive side.

Glorot and He initialization

- Each layer is characterized by the number of inputs, fan_{in} and outputs, fan_{out}

$$fan_{avg} = \frac{fan_{in} + fan_{out}}{2}$$

- For sigmoid/tanh/softmax activation function and normal distribution of input weights

$$\sigma^2 = \frac{1}{fan_{avg}}$$

- For sigmoid activation function and uniform distribution in the range $-r$ and r

$$r = \sqrt{\frac{3}{fan_{avg}}}$$

Why initialization matters

- Bad initialization can make a deep network fail before learning even really starts.
- Good initialization gives every layer a usable starting point, so gradients remain more stable.
- This does not solve everything, but it often makes training dramatically easier.

Choosing initialization in practice

- For sigmoid or tanh, Glorot initialization is the usual default.
- For ReLU, Leaky ReLU, ELU, GELU, Swish, and Mish, He initialization is usually preferred.
- For SELU, LeCun initialization together with other SELU conditions.

Keras example: He initialization

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(
        50,
        activation="relu",
        kernel_initializer="he_normal")
])
```

Better activation functions

- Activation functions affect how easily gradients travel through the network.
- Modern deep learning usually avoids plain sigmoid in hidden layers and prefers ReLU-like functions.

ReLU as a practical default

- ReLU outputs 0 for negative inputs and keeps positive inputs unchanged.
- It is computationally cheap and avoids saturation on the positive side.
- Because of this, ReLU became the default hidden-layer activation in many deep models.

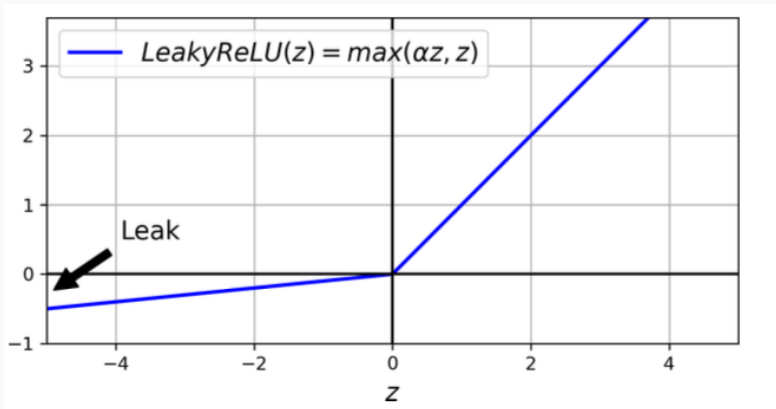
The dying ReLU problem

- A ReLU neuron outputs zero whenever its input stays negative.
- If this happens for all training instances, the neuron may stop learning completely.
- Large learning rates make this problem more likely, because one update can push a unit into that state.

Leaky ReLU

- Leaky ReLU keeps a small slope for negative inputs instead of returning exact zeros.
- This means neurons can still receive gradients even when their input is negative.
- In practice, it often performs better than plain ReLU and reduces the risk of dead units.

Leaky ReLU



PReLU and RReLU

- PReLU learns the negative slope parameter from data instead of fixing it by hand.
- RReLU samples the slope randomly during training, adding a mild regularization effect.
- These variants can help, but they also add complexity and may overfit on small datasets.

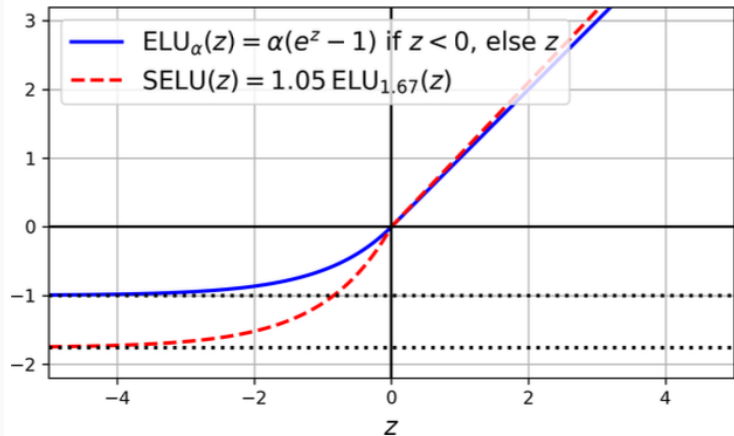
Keras example: Leaky ReLU

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(
        50,
        kernel_initializer="he_normal"),
    tf.keras.layers.LeakyReLU(alpha=0.2)
])
```

- ELU behaves like ReLU for positive values, but returns smooth negative values for negative inputs.
- Its outputs are closer to zero on average, which helps reduce the vanishing gradient problem.
- It is usually more expensive to compute than ReLU, but it can train faster or generalize better.

- SELU is a scaled form of ELU designed to make activations self-normalize in deep MLPs.
- If the required conditions are satisfied, each layer tends to keep mean near 0 and standard deviation near 1.
- This can stabilize deep training, but it works best only in fairly specific settings.

ELU and SELU activation functions



SELU constraints

- Input features must be standardized before training.
- Hidden layers must use LeCun normal initialization, and the architecture should be a plain MLP.
- Several usual techniques, such as standard dropout or batch normalization, break the self-normalizing theory.

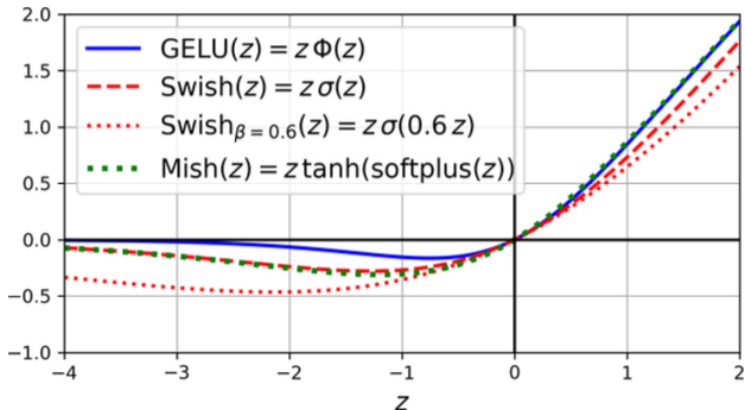
GELU, Swish, and Mish

- These are smooth modern activations that often outperform ReLU on harder tasks.
- They are not simple piecewise linear rules, so they can model more subtle behavior.
- The trade-off is that they require more computation than plain ReLU.

How to think about them

- GELU can be seen as a smooth gate that keeps or attenuates values depending on their magnitude.
- Swish multiplies the input by a sigmoid, creating a smooth nonmonotonic curve.
- Mish is similar in spirit and often performs very well, but usually by only a small margin.

GELU, Swish and Mish activation functions



Activation-function guidelines

- For simple tasks, ReLU remains a strong default because it is fast and well supported.
- For more complex tasks, Swish or GELU are often excellent choices.
- For deep plain MLPs, SELU can be worth trying if its conditions are respected.

Batch normalization: core idea

- Batch normalization normalizes layer inputs using statistics computed on the current mini-batch.
- After normalization, it also learns how to rescale and shift those values.
- This helps keep activations in a stable range during training.

Why batch normalization helps

- It reduces unstable gradients, so deeper models become easier to optimize.
- It makes training less sensitive to initialization and often allows larger learning rates.
- It also has a regularizing effect, sometimes reducing the need for other regularizers.

Training versus inference with BN

- During training, BN uses the mean and standard deviation of each mini-batch.
- During inference, it cannot rely on a mini-batch, so it uses moving averages collected during training.
- This is why BN behaves slightly differently during training and prediction.

BN parameters

- Each BN layer learns a scale parameter and a shift parameter for every input feature.
- It also maintains moving averages of the mean and variance seen during training.
- Some parameters are trained by backpropagation, while the moving statistics are updated separately.

Keras example: batch normalization

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28,
        28]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(300,
        activation="relu",
        kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(100,
        activation="relu",
        kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(10, activation="
        softmax")
])
```

Where to place BN

- BN can be placed before or after the activation function, and both designs are used in practice.
- If BN is placed before activation, the previous Dense layer often omits the bias term.
- The best choice can depend on the problem, so experimentation is often worthwhile.

Important BN hyperparameters

- Momentum controls how fast moving averages adapt during training.
- The axis parameter determines which dimensions are normalized together.
- These defaults usually work well, but unusual data layouts can require adjustment.

Limits of batch normalization

- BN makes each training step slower because it adds extra computations.
- It also depends on mini-batch statistics, so very small batches can make it less reliable.
- Even so, the faster convergence often makes total wall-clock training time shorter.

Gradient clipping

Gradient clipping

- Gradient clipping caps gradient values or norms during backpropagation.
- This prevents rare but dangerous updates from becoming extremely large.
- It is especially useful when exploding gradients are suspected, for example in recurrent networks.

Clipping by value versus norm

- Clip by value truncates each gradient component independently to a fixed interval.
- Clip by norm rescales the whole gradient vector when its norm exceeds a threshold.
- Norm clipping preserves direction better, while value clipping is simpler and often still works well.

Keras example: gradient clipping

```
optimizer = tf.keras.optimizers.SGD(clipvalue  
    =1.0)  
model.compile(loss="  
    sparse_categorical_crossentropy",  
               optimizer=optimizer,  
               metrics=["accuracy"])
```

Transfer learning idea

- Instead of training a large model from scratch, we can start from a model trained on a related task.
- Lower layers often learn general patterns, while upper layers are more task-specific.
- So it is common to keep lower layers and replace only the task-specific top part.

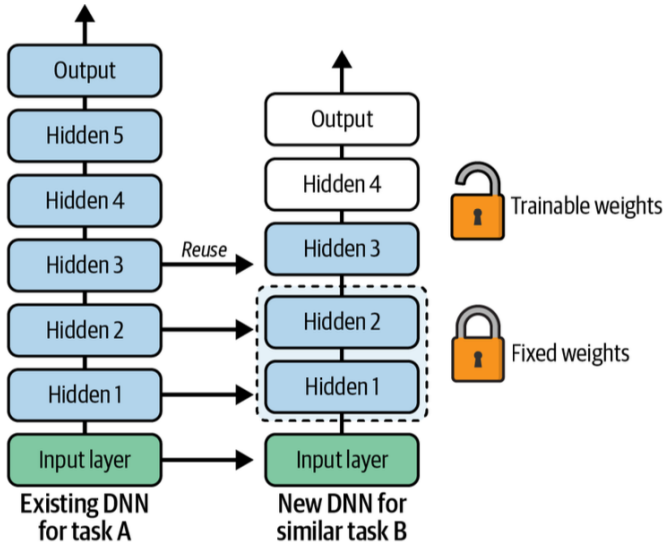
When transfer learning works best

- It works best when the old task and the new task are similar.
- It is particularly effective with deep convolutional networks, because early visual features are reusable.
- It is often less effective with small dense networks, whose learned patterns are more specific.

How to fine-tune safely

- A common strategy is to freeze reused layers at first and train only the new output layer.
- After that, some top reused layers may be unfrozen and fine-tuned with a smaller learning rate.
- This avoids destroying the useful weights learned on the original task.

Reusing pre-trained layers



Transfer learning workflow in Keras

```
for layer in model_B_on_A.layers[:-1]:  
    layer.trainable = False  
  
model_B_on_A.compile(  
    loss="binary_crossentropy",  
    optimizer=tf.keras.optimizers.SGD(  
        learning_rate=0.001),  
    metrics=["accuracy"])
```

Unsupervised pretraining

- Sometimes labeled data is scarce, but unlabeled data is plentiful.
- In that case, we can first train an unsupervised model to learn useful representations.
- Then we reuse part of that model and fine-tune it on the smaller labeled dataset.

Why this can help

- The unsupervised step teaches the network about the structure of the input data.
- This gives supervised training a better starting point than random initialization.
- Historically, this idea helped restart deep learning before modern training tricks became common.

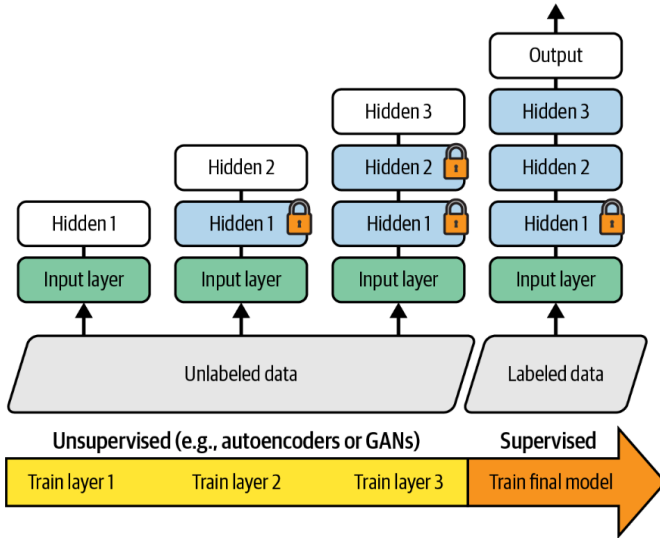
Autoencoders and GANs

- Common tools for unsupervised pretraining are autoencoders and GANs.
- After training one of these models, lower layers can be reused for the final supervised task.
- The final network is then fine-tuned with the labeled data available.

Greedy layer-wise pretraining

- Historically, researchers sometimes trained one layer at a time, freezing each layer before adding the next.
- This was useful when deep networks were much harder to optimize than they are now.
- Today, full unsupervised models are usually trained in one shot instead.

Unsupervised pretraining



Pretraining on an auxiliary task

- If no similar pretrained model exists, a model can be trained on a different but related task.
- The auxiliary task should be easy to label automatically or much cheaper to label.
- Its lower layers may then provide useful features for the final target task.

Self-supervised learning

- Self-supervised learning creates labels from the data itself, such as masking words in text.
- A model is trained to solve that automatically generated prediction task.
- The learned representation can then be reused for downstream supervised problems.

Faster optimizers

Faster optimizers

- Good initialization and activations help, but optimization choice also matters a lot.
- Modern optimizers can converge faster and with less manual tuning than plain gradient descent.

Momentum

- Momentum adds part of the previous update to the current one.
- This lets optimization build speed in directions that consistently reduce the loss.
- As a result, it moves faster through flat regions and reduces oscillations in narrow valleys.

Physical intuition for momentum

- Consider a ball rolling down a slope and gaining velocity.
- Without momentum, gradient descent reacts only to the current slope and forgets the past.
- With momentum, previous gradients influence the next move, often improving convergence a lot.

Momentum algorithm

$$\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta)$$

$$\theta \leftarrow \theta + \mathbf{m}$$

- At each iteration, the momentum is updated by subtracting the local gradient
- The algorithm introduces a hyperparameter β , called momentum

Keras example: momentum

```
optimizer = tf.keras.optimizers.SGD(  
    learning_rate=0.001,  
    momentum=0.9)
```

Nesterov accelerated gradient

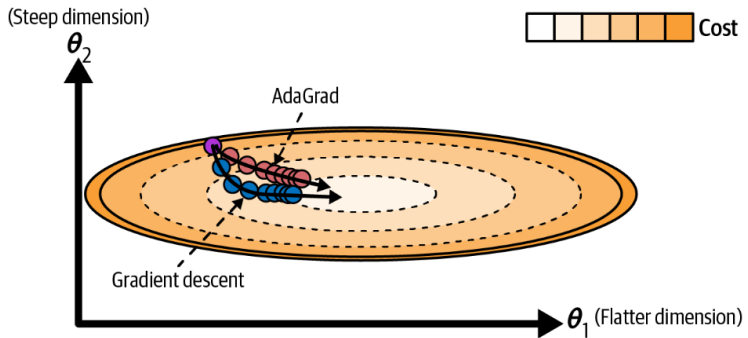
- Nesterov momentum is a refined version that looks ahead before computing the gradient.
- This often corrects direction earlier and reduces oscillations more effectively.
- In practice, it is usually obtained by simply turning on a Nesterov option in the optimizer.

Keras example: Nesterov

```
optimizer = tf.keras.optimizers.SGD(  
    learning_rate=0.001,  
    momentum=0.9,  
    nesterov=True)
```

- AdaGrad adapts the learning rate separately for each parameter.
- Parameters receiving large gradients get smaller future steps, while others keep larger steps.
- This helps on some problems, but it often slows down too much for deep neural networks.

Figure 11-8



RMSProp

- RMSProp fixes AdaGrad's tendency to stop too early by using only recent squared gradients.
- This keeps adaptive learning rates, but avoids the continual shrinkage that harms AdaGrad.
- It became one of the most influential optimizers for neural network training.

Adam and variants

- Adam combines momentum with adaptive learning rates, so it usually works well with little tuning.
- Variants such as AdaMax, Nadam, and AdamW refine different parts of the same idea.
- Because of its reliability, Adam is often the first optimizer practitioners try.

Optimizer comparison

- Plain SGD is simple but often slower and harder to tune.
- Momentum and Nesterov usually improve SGD substantially with little extra complexity.
- For many deep-learning tasks, RMSProp or Adam-family optimizers are strong practical choices.

Learning rate matters enormously

- If the learning rate is too high, training may diverge or oscillate wildly.
- If it is too low, training may converge but waste a huge amount of time.
- A good schedule often works better than any single constant learning rate.

Figure 11-9



Common learning schedules

- Power scheduling gradually reduces the learning rate according to the iteration count.
- Exponential scheduling repeatedly multiplies the learning rate by a fixed decay factor.
- Piecewise constant scheduling uses a few manually chosen plateaus separated by drops.

Performance scheduling and 1cycle

- Performance scheduling lowers the learning rate when validation performance stops improving.
- The 1cycle strategy first raises the learning rate, then lowers it strongly later in training.
- This schedule can train faster and sometimes reach better generalization.

Keras example: scheduled learning rate

```
def exponential_decay_fn(epoch):  
    return 0.01 * 0.1 ** (epoch / 20)  
  
lr_scheduler = tf.keras.callbacks.  
    LearningRateScheduler(  
        exponential_decay_fn)
```


Regularization in deep nets

- A large network can memorize training data instead of learning patterns that generalize.
- Regularization techniques reduce this risk by limiting complexity or introducing noise.

L1 and L2 regularization

- L2 regularization discourages large weights and often improves generalization smoothly.
- L1 regularization pushes many weights toward zero and can produce sparse models.
- These penalties are added to the loss during training and controlled by hyperparameters.

Dropout

- Dropout randomly disables some units during training, forcing the network not to depend too much on any one path.
- This acts like noise injection and often reduces overfitting.
- At inference time, all units are used, but activations are appropriately scaled.

Monte Carlo dropout

- If dropout is kept active at prediction time, multiple forward passes can produce a distribution of outputs.
- This gives a practical way to estimate predictive uncertainty.
- So dropout can be useful not only for regularization, but also for uncertainty estimation.

Max-norm regularization

- Max-norm places an upper bound on the norm of incoming weights for each neuron.
- This prevents weights from growing excessively large.
- It is another way to stabilize training and limit overfitting.

Training sparse models

- Sometimes a compact sparse model is preferable for speed or memory reasons.
- Strong L1 regularization can encourage many weights to become zero.
- Pruning tools can also remove weak connections during or after training.

Practical advice

- Start with sensible defaults: good initialization, a strong activation, and a modern optimizer.
- Use batch normalization or other stabilization tools when networks become deep or training becomes erratic.
- When data is scarce, prefer reuse of pretrained knowledge over training everything from scratch.

A beginner-friendly recipe

- Standardize inputs when appropriate, choose ReLU or Swish-like activations, and initialize weights correctly.
- Train with Adam or momentum SGD, monitor validation performance, and use a learning-rate schedule.
- Add regularization only as needed, because too much regularization can also hurt learning.